

Практикум 2.1 — «Тензоры и операции (NumPy)»

Методические указания для студентов

Цели

- Понять понятия ранг тензора, форма, тип данных (dtype).
- Научиться выполнять индексацию, срезы, reshape, transpose и broadcasting.
- Сравнить наивные циклы и векторизацию NumPy по времени выполнения.

Задания

- 1) Создайте скаляр, вектор, матрицу и 3D-тензор. Выведите их ранги и формы.
- 2) Для матрицы A (4×6) получите фрагменты срезами: верхние 2 строки, последние 3 столбца; транспонируйте и объясните форму.
- 3) Реализуйте функции naive_add(x, y), naive_relu(x) и naive_matrix_vector_dot(M, v). Сравните скорость с x+y и M.dot(v) с помощью %timeit.
- 4) Мини-проект: нормализуйте сгенерированное «изображение» 64×64 (см. шаблон ноутбука). Постройте картинки «до/после».

Критерии оценивания

Критерий	Макс. балл	Отметки преподавателя
Корректная работа с формами и срезами	20	
Реализация naive_* и сравнение с векторизацией	30	
Мини-проект «изображение»: корректность и пояснения	30	
Оформление ноутбука и выводы	20	

Шаблон отчёта (кратко)

1. Цель работы.
2. Ход выполнения (код/скриншоты/комментарии).
3. Сравнение времени (таблица или вывод %timeit).
4. Выводы (что узнали, где пригодится).

Практикум 2.1 — Тензоры и операции (NumPy)

Цели: формы, срезы, reshape/transpose, broadcasting, векторизация.

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42) # для воспроизводимости
```

Задание 1: Создание тензоров и их свойства

```
In [ ]: x = np.array(12)
v = np.array([1,2,3])
M = np.array([[1,2,3],[4,5,6]])
T = np.random.randn(2,3,4)
print('ranks:', x.ndim, v.ndim, M.ndim, T.ndim)
print('shapes:', M.shape, T.shape)
print('dtype T:', T.dtype)
```

Задание 2: Индексация, срезы, reshape/transpose

```
In [ ]: A = np.arange(24).reshape(4,6)
print(A)
print('slices:\n', A[:2, 3:])
B = A.reshape(6,4)
C = A.T
print('B.shape =', B.shape, ' C.shape =', C.shape)
u = np.array([1,2,3,4,5,6])
print('A+u:\n', A+u) # broadcasting
```

Задание 3: Векторизация vs наивные циклы

```
In [ ]: def naive_add(x, y):
    assert x.shape == y.shape
    z = np.empty_like(x)
    for i in range(x.shape[0]):
        z[i] = x[i] + y[i]
    return z

def naive_relu(x):
    z = np.empty_like(x)
    for i in range(x.shape[0]):
        z[i] = x[i] if x[i] > 0 else 0
    return z

def naive_matrix_vector_dot(M, v):
```

```

assert M.shape[1] == v.shape[0]
res = np.zeros(M.shape[0])
for i in range(M.shape[0]):
    s = 0.0
    for j in range(M.shape[1]):
        s += M[i, j] * v[j]
    res[i] = s
return res

x = np.random.randn(100000)
y = np.random.randn(100000)
# %timeit naive_add(x, y)
# %timeit x + y
M = np.random.randn(1000, 1000)
v = np.random.randn(1000)
# %timeit naive_matrix_vector_dot(M, v)
# %timeit M.dot(v)

```

Задание 4 (мини-проект): простая обработка «изображения»

```

In [ ]: img = np.random.randint(0,255,size=(64,64)).astype('float32')
img_norm = (img - img.mean()) / (img.std() + 1e-8)
img_01 = (img - img.min()) / (img.max() - img.min() + 1e-8)
img_boost = np.clip(img_01 * 1.5, 0, 1)
print(img.shape, float(img_norm.mean()), float(img_norm.std()))
plt.figure(); plt.imshow(img, cmap='gray'); plt.title('Original'); plt.axis('off')
plt.figure(); plt.imshow(img_boost, cmap='gray'); plt.title('Boosted'); plt.axis('o

```

Выводы

Опишите наблюдения по формам, времени выполнения, качеству изображения после нормализации.